NafaPilot

Profit-aware merchant growth decision engine

Build report: what was built in Milestones 1–9, why, and how to run it

Team PayMatrix · Paytm Build for India AI Hackathon 2026 · Track 1: Merchant Growth AI

About this document. It explains each milestone in plain language: the goal, what was built, how it works, why those choices were made, and how it was verified. Chapter 13 lists every command needed to run the backend and frontend on Windows. Chapter 14 shows how to connect hosted services (Neon or Aiven PostgreSQL, Render, Vercel, Gemini).

Prototype. All merchants, orders and outcomes are synthetic, and campaign execution is simulated. This is an independent hackathon submission, not an official Paytm product, and it has no access to Paytm systems. Status as of 29 October 2026.

Contents

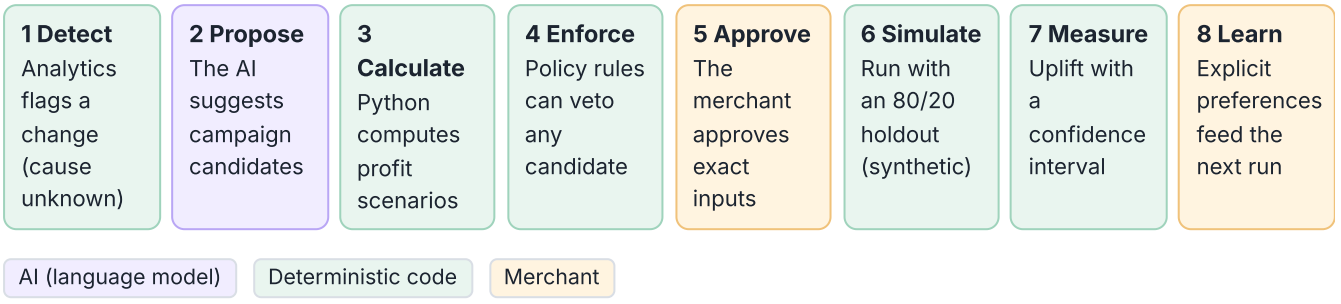
1. The big picture
2. Technology choices and why
3. Architecture and data flow
4. M1 Project setup
5. M2 LLM provider (Gemini + mock)
6. M3 Synthetic dataset
7. M4 Analytics and baseline engine
8. M5 Profit and policy engines
9. M6 Database and campaign API
10. M7 Recommendations and explanations
11. M8 Simulated execution and measurement
12. M9 Merchant dashboard
13. Running it: every command
14. Integrating other services
15. Troubleshooting
16. Tests and quality checks
17. Limitations and next step
18. Glossary

1. The big picture

A small merchant (a café, a kirana store) sees a change in sales, such as fewer weekday-afternoon orders. NafaPilot helps them decide whether a campaign is worth running, and then measures whether it actually worked.

Most "AI marketing" tools optimise for *more orders*. NafaPilot optimises for **incremental profit**: the extra profit a campaign causes after paying for the discount, the campaign cost, and the discount given away on orders that would have happened anyway. That last item (called **S**, cannibalisation) is the one most tools ignore, and it is often what makes a "successful" campaign lose money.

The workflow



> **The core principle.** *Gemini proposes; Python calculates; policy enforces; the merchant authorises; the experiment engine measures.* Chapter 2 lists the four kinds of numbers this produces and how they are kept apart. The language model never produces an authoritative number and cannot override a policy rule. Every profit figure, every pass/fail and every verdict comes from deterministic, tested code. This matters because language models can produce confident but wrong arithmetic; money decisions need reproducible calculations that a judge or auditor can check by hand.

2. Technology choices and why

Choice	Why it was chosen
Python + FastAPI (backend)	Python has the best data and statistics libraries (pandas, NumPy, SciPy). FastAPI generates interactive API docs at `/docs` automatically, validates every request with Pydantic, and is simple to deploy on Render.
Pydantic v2	Every request, response and LLM output is a typed model. Bad input is rejected with a clear message instead of causing a wrong calculation. LLM output that doesn't match the schema is rejected, never "repaired".
Decimal for money	Floating point can't represent ₹0.10 exactly; repeated sums drift. Python's `Decimal` gives exact paise arithmetic, so the deck's numbers (+₹100, +₹2,400, +₹4,700) reproduce exactly.
pandas, NumPy, SciPy	Order aggregation (pandas), seeded random simulation (NumPy), and well-tested statistical tests (SciPy: exact binomial test, Welch's t-interval). Using a standard library rather than hand-written statistics reduces the chance of errors.
PostgreSQL 16 + SQLAlchemy 2 + Alembic	A real relational database with transactions, foreign keys, CHECK constraints and triggers. These are used to *enforce* rules (an approval must match the exact version, history can't be edited, a version runs once) at the database level, not only in code. Alembic gives versioned, reversible schema migrations. Hosted PostgreSQL (Neon, Aiven, Render) needs no code change.
SQLite (tests only)	Every database test also runs on a throwaway SQLite file, so tests work without a server. The same tests run on PostgreSQL when `TEST_DATABASE_URL` is set.
Google Gemini via `google-genai`	The hackathon context and the team's choice. The model is configurable (`GEMINI_MODEL`), currently `gemini-3.8-flash` . It is used only to propose candidates and write explanations.
Mock LLM provider	A deterministic, offline stand-in for Gemini. The whole app runs without an API key, demos work without internet, and tests are repeatable and free.
Next.js 16 + React 19 + TypeScript + Tailwind v4 (frontend)	Next.js is the standard React framework and deploys easily to Vercel. TypeScript types mirror the backend schemas, so a field mismatch is caught at build time. Tailwind keeps styling consistent without a heavy design system. Charts are hand-written SVG, so there are no chart-library dependencies.
pytest, ruff, mypy --strict	Tests, linting and strict static typing on the backend. Strict typing catches whole classes of bugs (wrong field names, None where a value is required) before running.
Vitest + Testing Library	The testing setup recommended by the Next.js docs. Tests run in a simulated browser (jsdom) and need neither the backend nor Gemini.
Docker Compose (optional)	One command starts a local PostgreSQL 16 with the right user and database.

Four kinds of numbers the system keeps apart

Kind	Meaning	Where it comes from (API label)
Observed	What happened in the (synthetic) order history	M4 baseline
Projection	"If the latest rate continues for 4 weeks." Not observed, not a forecast	M4 alerts (`is_projection`)
Prediction	Profit under assumed order counts, in three scenarios	M5 profit engine (`result_kind`)
Measured (simulated)	Treatment vs holdout estimate with uncertainty, from simulated outcomes	M8 experiments (`is_simulated`)

> **Deliberately not used:** microservices, agent frameworks (LangChain etc.), model training or fine-tuning, and any call to Paytm or a real messaging/payment service. The project requirements excluded them, and they would add complexity without making the decisions more correct.

3. Architecture and data flow

One FastAPI backend and one Next.js frontend. The backend is organised in layers; inner layers never depend on outer ones.

Layer	Folder	Responsibility
API routes	`backend/app/api/v1/`	HTTP endpoints. Thin: validate input, call a service, return a schema.
Schemas	`backend/app/schemas/`	Pydantic request and response models (the API contract).
Engines (pure)	`services/analytics`, `services/decision`, `services/experiments`	Calculations only. No database, no files, no LLM. Same input, same output. Easy to test.
Workflow services	`services/campaign_store.py`, `recommendations.py`, `execution.py`	Transactions, refusals, audit events. Call the engines and the database.
LLM layer	`services/llm/`	Provider interface, Gemini, mock, prompts, grounding check, templates. The engines never import it.
Data access	`backend/app/data/`	Reads the synthetic CSV dataset (merchants, orders, customers, policies).
Database	`backend/app/db/`, `backend/migrations/`	SQLAlchemy models and Alembic migrations 0001–0003.
Frontend	`frontend/`	Typed API client (`lib/api.ts`), pages and components. Renders what the API returns; calculates nothing authoritative.

Two places data lives

Synthetic dataset (CSV files)
`data/synthetic/`, generated by `scripts/generate_dataset.py`. Read-only input: merchants, policies, customers, orders, past campaigns and their outcomes. Not committed to Git and not in the zip; generate it after extracting.

PostgreSQL database
Wherever `DATABASE_URL` points (local by default). Everything the app *creates*: campaigns, versions, evaluations, approvals, recommendation runs, explanations, preferences, executions, assignments, outcomes, measurements, audit events.

One request, end to end (example: run a simulation)

1. The dashboard creates a fresh `Idempotency-Key` and calls `POST /api/v1/campaigns/{id}/versions/1/execute`.
2. The route passes the request to `execution.py`, which writes an `execution_requested` audit event.
3. In one locked transaction it checks eligibility: current version, approved for the exact input hash, policy still passes, not already executed.
4. It reads eligible customers from the CSV dataset, splits them 80/20 with a deterministic hash, simulates outcomes, and measures uplift with SciPy.

5. It writes the execution, assignments, outcomes, measurement and audit events, and marks the campaign `simulated`, all in the same transaction.
6. It returns the experiment; the dashboard navigates to the results page.

M1 Project setup

A working skeleton that the rest of the project can grow into

What was built

- **Repository layout**: `backend/` (FastAPI), `frontend/` (Next.js), `scripts/`, `data/`, `docs/`, `docker-compose.yml`, `.env.example`, `README.md`.
- **Backend skeleton**: `app/main.py` with `GET /health`, a versioned router at `/api/v1`, and `GET /api/v1/meta`, which labels the data as synthetic and execution as simulated.
- **Configuration**: `app/config.py` reads settings from environment variables and the root `.env` with `pydantic-settings`. The Gemini key is a `SecretStr`, so it never appears in logs or `repr()`.
- **CORS**: only the origins in `CORS_ORIGINS` (default `http://localhost:3000`) may call the API from a browser.
- **Logging with redaction**: a log filter scrubs any configured secret from log messages and arguments.
- **Frontend skeleton**: Next.js with TypeScript and Tailwind, a status page that checks the backend connection. System fonts are used so the build works offline.
- **Docker Compose**: PostgreSQL 16 with user, password and database `nafapilot`, a health check and a persistent volume.
- **Git hygiene**: `.gitignore` excludes `.env`, virtual environments, `node_modules`, build output, local database files and generated data.

> **Why.** Settling configuration, secrets handling and the folder structure first means later milestones only add code; they never restructure. Putting secrets handling in from day one (SecretStr, redaction, git-ignored `.env`, nothing secret in the frontend) is much safer than retrofitting it.

M2 LLM provider foundation (Gemini + mock)

A safe, swappable way to talk to the language model

What was built

- **Provider interface** (`services/llm/base.py`): three methods, `generate_candidates` , `explain` and `ping` . The rest of the app depends only on this interface.
- **Gemini provider** (`gemini.py`): uses the `google-genai` SDK with structured (JSON-schema) output. A thin `GeminiTransport` is the only code that touches the SDK, so tests replace it with a fake and never make network calls.
- **Mock provider** (`mock.py`): deterministic, offline. It mirrors the deck's illustrative candidates (a targeted offer, an off-peak offer, no action), scaled to the merchant's order value and discount cap.
- **Strict validation**: every model response is validated against Pydantic schemas (`schemas/llm.py`). Invalid output raises `llm_invalid_response` ; it is never patched up.
- **Typed errors** with stable codes and HTTP statuses: `llm_not_configured` (503), `llm_quota_exceeded` (429), `llm_timeout` (504), `llm_upstream_error` (502), `llm_auth_or_request_error` (502), `llm_invalid_response` , `llm_safety_blocked` .
- **Retries** with backoff for transient errors (`LLM_MAX_RETRIES` , default 2) and a timeout (`LLM_TIMEOUT_SECONDS` , default 20).
- **Endpoints**: `GET /api/v1/llm/status` reports the provider, model and whether a key is present (never the key). `POST /api/v1/llm/ping` does a live round-trip (disabled in production).
- **Script**: `scripts/check_gemini.py` lists available models and runs a sample end to end.

> **Why an interface plus a mock.** Tests must never depend on a paid, rate-limited network service, and a demo must not fail because of Wi-Fi or quota. With a mock behind the same interface, the full workflow runs offline, and switching to Gemini is one environment variable (`LLM_PROVIDER=gemini`).

> **Why structured output and strict validation.** Free text from a model can't be safely turned into numbers. A JSON schema plus Pydantic validation means only well-formed candidates (type, discount, target, assumed order counts) reach the profit engine. Rationale text is kept for display but never used in calculations.

How the key is protected

- Stored only in the backend `.env` (git-ignored) or the hosting platform's environment variables.
- Passed only to the SDK client. Upstream error messages are redacted before use.
- Never sent to the frontend, never returned by the API, never written to logs or audit events.

M3 Synthetic dataset generator

Realistic, reproducible data with known answers

What was built

- **Generator**: `backend/app/synthetic/` plus `scripts/generate_dataset.py`. Default seed 42; `--seed` and `--out` options.
- **Reproducible**: the same seed produces byte-identical CSV files on any OS. `manifest.json` records the seed, row counts and a SHA-256 hash of every file.
- **Tables**: 20 merchants, merchant policies, customers, about 12 weeks of orders (3 Aug–25 Oct 2026), past campaigns with holdout assignments and outcomes, merchant feedback, and a file of deliberately broken orders for validation tests.
- **Ground truth**: past campaigns record their true base rate, true lift and true expected profit, plus an `outcome_label` (profitable, unprofitable, loss-limit violation, null effect, discount-cap breach). Tests check that NafaPilot reaches the same verdict.

Planted scenarios (each tests a specific behaviour)

Merchant / record	Scenario	Tests
M001 Anand Café	Weekday 2–5 pm orders: 38/day for 4 weeks, then falling to 30 (−21%). AOV ₹200, cost ₹110, cap 15%, loss limit ₹1,000, holdout 20%	Opportunity detection; matches the deck's slide 2
C001 (M001)	₹20 targeted offer, 240 treated / 60 held out	Experiment measurement
C002–C005	Discount above cap; small loss; large loss; zero true lift	Policy breach, loss limit, null effect
M007	No variable cost recorded	`insufficient_data` for profit
M011	30% of orders lack cost, 60% lack customer ID	Data-quality warnings
M015	Only 3 weeks of history	Too little history to compare periods
M019	10 consecutive days with no orders	Reporting gaps without guessing their cause
Feedback rows	Rejections with and without reasons; an edit; a strategic "festival rush" rejection	Preference learning without over-interpreting free text

Why synthetic data with planted scenarios. Real merchant data isn't available, and using it would raise privacy issues. Synthetic data with *known truth* lets the tests prove the engines are right: if the generator planted a −21% decline, the analytics must find it; if a campaign's true lift is zero, the experiment must not claim a win. Edge cases (missing costs, gaps, short history) are planted deliberately so the code is forced to handle them honestly.

Everything in `data/synthetic/` is invented. It shows the software works; it is not evidence about real merchants or customers.

M4 Analytics and baseline engine

What happened, how reliable the data is, and what changed

What was built

- **Validation** (`analytics/validation.py`, `POST /api/v1/data/validate`): rows whose identity, merchant, time or value can't be trusted are rejected (errors); rows with a bad optional field are kept, and the bad value is treated as missing (warnings). Nothing is "corrected" or guessed. The report gives counts, examples and field completeness.
- **Baseline** (`analytics/baseline.py`): the last 28 calendar days. Orders, orders per day, average order value, revenue, contribution profit (from recorded costs, or the merchant setting, or "unavailable"), customer-ID coverage, repeat-customer share, a daily series and a weekday × hour heat map.
- **Coverage honesty**: the dataset records orders only; it has no calendar of opening days. A day with no orders could be a closure, an outage or genuinely zero sales. So rates are *per day with recorded orders*, and runs of 3+ empty days are reported as gaps with cause unknown.
- **Opportunity screening** (`analytics/opportunities.py`): segments (all hours, weekday and weekend time slots), weekly rates, reference = first 4 of the last 12 weeks, recent = last 2 weeks.
- **Projection**: for an alert, a "next 4 weeks if nothing changes" figure at the latest weekly rate. It is labelled `is_projection: true` with a caveat; it assumes trading every calendar day.
- **Endpoint**: `GET /api/v1/merchants/{id}/baseline` returns all of the above plus `method_note`.

How an alert is decided

1. For each segment, an **exact binomial test** compares recent and reference order counts, with days-with-orders as exposure.
2. **Benjamini–Hochberg** controls the false-discovery rate at 2.5% across the merchant's segments.
3. An **effect-size gate** requires a change of at least 10% in the same direction for both the latest week and the last two weeks.
4. A **confidence label** ("high" or "medium") is a rule: "high" when an 8-week linear trend agrees and no week has missing days. It is not a probability.
5. The **cause is always "unknown"**. Order counts can't explain why sales changed.

> **Why these choices.** A merchant has many time segments; testing each at $p < 0.05$ would raise false alarms by chance. An earlier version used a simple $p < 0.01$ cut-off; during the M4 review it was replaced with Benjamini–Hochberg because the naive threshold produced false alerts. The effect-size gate stops tiny but "significant" changes from being shown. Refusing to name causes avoids a common AI failure: inventing a plausible story ("a competitor opened nearby") from data that can't support it.

> **Stated limits.** The binomial test assumes Poisson-like orders; real orders vary more, so p-values are nominal and alerts are a screening aid. The synthetic data is Poisson, so it flatters the method. These caveats are in the code, the docs and the API response.

Review corrections (M4 audit)

After the independent review, targeted fixes were made to: coverage semantics (per day with recorded orders), validation rules (which problems reject a row vs. warn), the baseline date range, the statistical interpretation (nominal p-values, rule-based labels), and labelling the projected baseline as a projection.

M5 Profit and policy engines

Exact, explainable money calculations and hard limits

The profit formula

```
ΔR = I × (AOV − d) − S × d                 incremental net revenue
ΔP = ΔR − I × VC − C                       incremental contribution profit
I* = ceil((S × d + C) / (AOV − d − VC))    break-even incremental orders
```

Symbol	Meaning
I	Incremental orders: orders that would not have happened without the offer
S	Existing orders that get the discount anyway (cannibalisation)
d / AOV / VC / C	Discount per order / average order value / variable cost per order / campaign cost besides the discount

Worked example (Campaign A, conservative): ΔR = 30 × (200 − 20) − 50 × 20 = ₹4,400; ΔP = 4,400 − 30 × 110 − 1,000 = **+₹100**.

Campaign	d	C	(I, S) cons. / exp. / opt.	ΔP	Break-even
A targeted	₹20	₹1,000	(30,50) / (60,40) / (90,30)	+₹100 / +₹2,400 / +₹4,700	26
B off-peak	₹30	₹1,200	(60,200) / (120,160) / (180,140)	−₹3,600 / +₹1,200 / +₹5,400	100

Three scenarios show a range instead of one falsely precise number. Missing inputs give `insufficient_data`, impossible ones `invalid_input`; nothing is defaulted. Each result carries an `input_hash` (SHA-256 of the inputs).

The six policy rules

Rule	Blocks	Passes when
`valid_inputs`	recommendation	The profit engine returned `ok`
`discount_cap`	recommendation	d / AOV ≤ cap (15% for M001)
`min_post_discount_margin`	recommendation	(AOV − d − VC) / (AOV − d) ≥ minimum (25%)
`max_loss`	recommendation	ΔP in the risk scenario (conservative) ≥ −limit (−₹1,000)
`campaign_budget`	recommendation	C ≤ budget (₹5,000)
`merchant_approval`	execution	An approval exists for exactly this input hash

Campaign B is blocked by `max_loss`: its conservative scenario loses ₹3,600, more than the ₹1,000 limit, even though its expected profit is positive. Approval can never override a failed rule. Limits are inclusive.

> **Why.** Exact Decimal arithmetic makes every figure reproducible by hand. Counting the discount exactly once (inside ΔR) avoids the double-counting error that makes campaigns look worse or better than they are. Separating "permitted" (policy) from "executable" (policy plus approval) makes the approval step meaningful. The engines import no LLM or database code, and a test checks this: an AI-suggested Campaign B whose rationale claims it is "approved and safe" is still blocked.

> **Two interpretations flagged for review:** post-discount margin is measured on the price the customer pays; the campaign budget applies to C only (the discount's risk is governed by `max_loss`). Both are documented in `docs/decision-engine.md`.

Code: `backend/app/services/decision/`. Endpoint: `POST /api/v1/campaigns/evaluate` (stateless).

M6 Database and campaign API

Versions, approvals and an audit trail that can't be rewritten

Tables (migration 0001)

Table	Mutability	Purpose
`campaigns`	Mutable pointer	Identity, merchant, current version, execution status
`campaign_versions`	Append-only	Immutable inputs and their `input_hash`
`campaign_evaluations`	Append-only	The M5 economics and policy result for each version
`campaign_approvals`	Append-only	One decision per version, bound by a composite foreign key to the exact (version, hash)
`audit_events`	Append-only	Who did what, when, with explicit payload fields only

Workflow rules

- **Create** stores version 1 and evaluates it, in one transaction.
- **Edit** creates a new version; an earlier approval never carries over.
- **Decide** works only on the current version, with the hash the merchant reviewed. Refusals (409, audited): `stale_version`, `input_hash_mismatch`, `already_decided`, `policy_blocked`.
- **Approval is not execution**: `execution_status` stays `not_executed` until M8's explicit simulation request.

Append-only enforced three ways

1. No update or delete code paths exist.
2. An ORM guard raises `ImmutableRecordError` before any change is flushed.
3. Database triggers reject `UPDATE` and `DELETE` (PL/pgSQL on PostgreSQL, `RAISE(ABORT)` on SQLite).

> **Why.** An approval must mean "I approved *these exact numbers*". Binding it to a SHA-256 hash of the inputs, enforced by a foreign key, makes it impossible for an edited campaign to inherit an old approval. Immutable versions and an append-only audit log make the history trustworthy for review. Enforcing rules in the database, not just in Python, protects against bugs and manual edits.

Code: `backend/app/db/`, `services/campaign_store.py`, `migrations/`. Endpoints: `/api/v1/campaigns` (create, list, get, versions, decision, audit).

M7 Recommendations, feedback and explanations

The AI proposes; code checks, evaluates and selects

Who decides what

Step	Done by
Observed facts (e.g. −21% afternoon decline, cause unknown)	M4 analytics
Candidate campaigns (type, discount, target, assumed I/S, rationale)	LLM (Gemini or mock)
Preference filters	Deterministic code
Profit scenarios, break-even, policy pass/fail	M5 engines
Which option is recommended	A fixed rule in code
Explanation wording	LLM, only if grounded; otherwise a template
Approve / reject	The merchant

Selection rule: among candidates that pass every policy rule, pick the highest expected profit; ties go to the higher conservative profit, then the smaller discount. If none has positive expected profit, recommend doing nothing.

Campaign cost is never invented: the merchant enters it per campaign type in the request. A type without a cost is `insufficient_data`.

Grounded explanations

An AI explanation is kept only if every number in its text appears in the records it was given ("2,400", "2400" and "2400.00" all match). Otherwise a deterministic template is stored with a `fallback_reason`. The database enforces `source = 'template' OR grounded`. Explanations are available in English and Hinglish, and the merchant can ask a follow-up question.

Feedback and learning

- Preferences change **only** when the merchant explicitly picks one while rejecting: `target`, `max_discount_inr` or `avoid_campaign_type`.
- Free-text reasons are stored as history but never turned into rules, by code or by the AI.
- Preferences are sent to the AI as context *and* enforced as filters in code.
- Every run says `model_retrained: false`. "Learning" means preferences and decision history, not training.

> **Why.** Language models are good at ideas and wording, unreliable at arithmetic and prone to inventing numbers. So the AI only suggests and explains, the numeric check rejects invented figures, and a fixed rule makes the selection auditable. Not inferring preferences from free text prevents a single remark ("busy festival week") from becoming a permanent constraint.

Code: `services/recommendations.py`, `services/llm/{grounding,templates}.py`; migration 0002. Endpoints: `POST /merchants/{id}/recommendations`, `GET /recommendations/{run_id}`, `POST /recommendations/{run_id}/explanations`, `GET /merchants/{id}/preferences`.

M8 Simulated execution and measurement

Did the campaign cause extra orders, and did it make money?

> No offer is sent and no real system is contacted. Outcomes are simulated from the synthetic dataset, so a positive result does not show that a real campaign would work.

Eligibility to run (one locked transaction)

A version runs only if it is the current version, has an **approved** decision bound to its hash, still passes every policy rule under the merchant's current policy, and has not run before. Refusals: `not_approved`, `stale_version`, `policy_blocked`, `already_executed`, `idempotency_key_reused`, `insufficient_data`. Each refusal is audited.

Idempotency (no duplicate runs)

- The caller sends an `Idempotency-Key`. A retry with the same key returns the original result (`replayed: true`); nothing is re-simulated.
- The service locks the campaign row and checks for existing runs first.
- The database has UNIQUE constraints on version and on key, so even concurrent requests can't create two runs (tested with 4 parallel threads on PostgreSQL).

Assignment (who gets the offer)

- Seed = SHA-256 of the version ID and input hash; the same version always gets the same split.
- Eligible customers are ordered by a hash; the first 20% (the policy's `holdout_pct`, rounded half-up) are the holdout. 1,109 eligible customers → 887 treatment / 222 holdout.
- Customers without an ID, duplicate IDs and anonymous walk-in orders are counted and reported, never given invented IDs.
- Holdout customers can never redeem: a database CHECK enforces it.

Outcome simulation

Per customer over 28 days: base orders ~ Poisson(base rate); treated customers get extra orders ~ Poisson(lift). The base rate is calibrated from the merchant's past synthetic holdout groups (merchants without history need it supplied). By default the true lift comes from the version's expected scenario, and a caveat says agreement with the prediction is then *by construction*. Missing outcomes are stored as NULL, never imputed.

Measurement

```
effect      = mean(treatment orders) − mean(holdout orders)    95% Welch t-interval (SciPy)
Î           = effect × treated customers
measured ΔP = Î × (AOV − VC) − R × d − C                    R = treatment redemptions
```

Verdicts: `positive` / `negative` (or `profitable` / `loss`) only if the whole interval is on one side of zero and each arm has at least 30 measured customers; otherwise `inconclusive` with a reason (`interval_includes_zero`, `small_sample`, `no_variation`, `insufficient_measured_outcomes`).

> **Why a holdout and an interval.** A campaign's "orders during the offer" include orders that would have happened anyway. Only comparing with a holdout that saw no offer isolates the *incremental* effect. A confidence interval shows how uncertain that estimate is, and refusing a verdict when the interval includes zero stops a lucky noise result from being presented as a win. Welch's test is standard and doesn't assume equal variances in the two groups.

> **Stated limits.** Welch is approximate for skewed counts; missing outcomes are assumed missing at random; the profit interval treats redemptions as fixed; one pre-specified metric is tested once (testing many would need a multiple-comparison correction). By design, about 5% of zero-effect runs still come out "positive" or "negative" by chance; an offline check of 400 null runs gave about 4%.

Code: `services/execution.py`, `services/experiments/`; migration 0003 (executions, assignments, outcomes, measurements; all append-only). Endpoints: `POST /campaigns/{id}/versions/{n}/execute`, `GET /campaigns/{id}/experiments`, `GET /experiments/{id}`, `GET /experiments/{id}/assignments`.

M9 Merchant dashboard

The whole workflow in a browser

Pages

Page	What it shows and does
`/`	Merchant list; backend and AI-provider status
`/merchants/{id}`	Last-28-day baseline with daily chart; coverage and data quality; detected opportunities with a labelled projection; policy and saved preferences; campaigns with status, predicted and measured profit side by side; the "Get recommendations" form (merchant enters campaign costs)
`/recommendations/{run}`	The rule's pick and why; each AI candidate with scenarios, break-even and policy results; explanations with their source; "explain again" in English or Hinglish
`/campaigns/{id}`	A 5-step progress rail; prediction; policy rules with reasons; inputs; edit as a new version; approve/reject (optional explicit preference); **Run simulation…**; version history and audit trail
`/experiments/{id}`	Backend verdicts; four separate blocks (prediction, observed, incremental effect, measured profit) with interval charts that always show zero; caveats; exclusions; simulation settings; paginated customer outcomes

Safety behaviours

- **Approval never runs anything.** The approve button only calls the decision endpoint (tested).
- **Run simulation…** (never "Launch campaign") opens a confirmation showing the merchant, version, predicted profit, holdout % and a synthetic-only notice. It's disabled with a reason when the version isn't approved, is policy-blocked or has already run.
- **Idempotency in the UI**: a fresh key for each new run; "Retry the same request" reuses the key after a network or server error; a refusal discards it. Double clicks send one request. Success appears only after the server confirms.
- **Honest display**: the backend's verdict is shown exactly; a positive point estimate with an interval crossing zero stays "inconclusive". Missing values show as "missing" or "no data", never 0.
- **Refusals** get a plain-language title and next step, with the raw code, message and details under "Technical details".

Kinds of numbers on screen

Prediction: dashed amber edge · Simulated: hatched violet edge · Estimate: solid slate edge · Observed history: grey edge · Projection: dotted edge. Each block also has a text tag, so colour isn't the only signal.

> **Why.** The dashboard must make the difference between recommendation, prediction, approval and execution obvious to a non-technical merchant and to judges. It calculates nothing authoritative; it renders the API's numbers, so the frontend can't drift from the tested engines. Accessibility basics (keyboard focus, labels, readable charts, mobile layout) are built in.

Code: `frontend/lib/api.ts` (typed client), `frontend/components/`, `frontend/app/`. Tests: `frontend/tests/` using real captured backend responses as fixtures.

13. Running it: every command (Windows PowerShell)

Prerequisites: Python 3.11+, Node.js 20+, and PostgreSQL 16 (Docker Desktop is easiest, or a local install, or a hosted database; see chapter 14). Run commands from the project folder (the one with `backend`, `frontend`, `scripts`).

Step 1. Environment file (once)

```
Copy-Item .env.example .env
# Edit .env if needed: DATABASE_URL, LLM_PROVIDER, GEMINI_API_KEY, CORS_ORIGINS
```

Step 2. Database (choose one)

```
# A) Docker: local PostgreSQL 16 with user/password/db = nafapilot
docker compose up -d
docker ps                          # should list nafapilot-db

# B) Local PostgreSQL install: create user nafapilot (password nafapilot) and database nafapilot

# C) Hosted (Neon/Aiven): put its URL in DATABASE_URL in .env (chapter 14)
```

Step 3. Backend (first terminal)

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt

python ..\scripts\generate_dataset.py          # REQUIRED: creates data\synthetic\ (seed 42)
python -m alembic upgrade head                  # create/upgrade tables (to 0003)
python -m alembic current                       # expect: 0003_simulated_execution (head)

python -m uvicorn app.main:app --reload --port 8000
```

Check: http://localhost:8000/health returns `{"status":"ok"}`; API docs at http://localhost:8000/docs; http://localhost:8000/api/v1/merchants returns 20 merchants.

If PowerShell blocks `Activate.ps1`: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (once).

Step 4. Frontend (second terminal)

```
cd frontend
Copy-Item .env.example .env.local              # NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
npm install
npm run dev                                     # http://localhost:3000
```

Step 5. Demo walk-through

1. Open http://localhost:3000 and pick **Anand Café**.
2. Under "Ask for recommendations", enter campaign costs (e.g. ₹1,000 targeted, ₹1,200 off-peak, ₹800 new-customer) and click **Get recommendations**.
3. Open the recommended option, review the prediction and policy, click **Approve version 1**.
4. Click **Run simulation...**, check the confirmation, then **Confirm and run simulation**.
5. Read the experiment results. Open option B to see a policy-blocked campaign.

Optional: use Gemini instead of the mock

```
# in .env (project root)
LLM_PROVIDER=gemini
GEMINI_API_KEY=<your key from Google AI Studio>
GEMINI_MODEL=gemini-3.8-flash

# check from backend\ with the venv active, then restart uvicorn
python ..\scripts\check_gemini.py --list-models
python ..\scripts\check_gemini.py
```

Other useful commands

```
python ..\scripts\generate_dataset.py --seed 7 --out ..\data\other   # another dataset
python -m alembic downgrade -1                  # undo the last migration (refused if simulated runs
exist)
npm run build; npm run start                    # production build of the frontend
docker compose down                             # stop the local database (data kept in its volume)
```

14. Integrating other services

No code changes are needed for any of these; everything is configured through environment variables.

Variable	Where	Purpose
`DATABASE_URL`	backend	PostgreSQL connection (must start with `postgresql+psycopg://`)
`CORS_ORIGINS`	backend	Comma-separated frontend URLs allowed to call the API
`LLM_PROVIDER`, `GEMINI_API_KEY`, `GEMINI_MODEL`	backend	`mock` or `gemini`, plus key and model
`ENVIRONMENT`	backend	`development` / `test` / `production` (production disables `/llm/ping`)
`DATA_DIR`	backend	Dataset folder; empty = `data/synthetic`
`NEXT_PUBLIC_API_BASE_URL`	frontend	Backend URL. Public by design: never put a secret in a `NEXT_PUBLIC_` variable

Neon (hosted PostgreSQL)

1. Create a project at neon.tech and copy the **direct** (not "pooled") connection string.
2. Change the prefix `postgresql://` to `postgresql+psycopg://` and keep `?sslmode=require`.
3. Put it in `.env` and run the migrations:

```
DATABASE_URL=postgresql+psycopg://USER:PASSWORD@ep-xxxx.REGION.aws.neon.tech/neondb?sslmode=require

python -m alembic upgrade head
python -m alembic current
```

Aiven (hosted PostgreSQL)

1. Create a PostgreSQL service and copy its Service URI (it uses a non-standard port).
2. Change `postgres://` to `postgresql+psycopg://`, keep `?sslmode=require`.
3. Run `python -m alembic upgrade head`.

```
DATABASE_URL=postgresql+psycopg://avnadmin:PASSWORD@SERVICE.aivencloud.com:PORT/defaultdb?
sslmode=require
```

Render (backend hosting)

Service type	Web Service, Python
Root directory	`backend`
Build command	`pip install -r requirements.txt && python ../scripts/generate_dataset.py`
Pre-deploy command	`alembic upgrade head` (or run it once from the Render shell, or locally against the same `DATABASE_URL`)
Start command	`uvicorn app.main:app --host 0.0.0.0 --port $PORT`
Environment	`DATABASE_URL` (Neon, Aiven or Render PostgreSQL), `ENVIRONMENT=production`, `CORS_ORIGINS=https://your-frontend.vercel.app`, `LLM_PROVIDER`, `GEMINI_API_KEY`, `GEMINI_MODEL`

The dataset is generated during the build because the CSVs aren't committed. The same seed gives identical data on every deploy.

Vercel (frontend hosting)

Root directory	`frontend`
Framework preset	Next.js (build `npm run build`)
Environment	`NEXT_PUBLIC_API_BASE_URL=https://your-backend.onrender.com` (set before building; it is baked into the bundle)
Then	Add the Vercel URL to the backend's `CORS_ORIGINS` and redeploy the backend.

> **Security rules for any service.** Keep keys and database passwords only in `.env` (git-ignored) or the platform's environment settings. Never paste them in chat, commit them, or put them in a `NEXT_PUBLIC_` variable. Never point `TEST_DATABASE_URL` at a real database: the PostgreSQL tests drop and recreate its schema. On Render's free tier the service sleeps when idle; the first request after a pause can take a while.

15. Troubleshooting

Symptom	Cause and fix
`/api/v1/merchants` returns **503**; error code `dataset_missing`	The synthetic CSVs don't exist (they're not in the zip or in Git). Run `python ..\scripts\generate_dataset.py` from `backend\`, then restart uvicorn.
Frontend says "Backend not reachable"	Backend not running, wrong `NEXT_PUBLIC_API_BASE_URL` in `frontend\.env.local`, or the frontend URL isn't in `CORS_ORIGINS`. Restart `npm run dev` after editing `.env.local`.
`alembic upgrade` fails to connect	The database isn't running (`docker compose up -d`) or `DATABASE_URL` is wrong. Hosted databases need `?sslmode=require`.
Recommendations return `llm_not_configured` (503)	`LLM_PROVIDER=gemini` without a key or model. Set them, or use `LLM_PROVIDER=mock`.
Every candidate is "insufficient data"	No campaign costs were entered in the recommendation form. Enter a cost for each type you'd consider.
Simulation refused: `insufficient_data`	Too few eligible customers (needs 30 per group) or no past holdout data (M007, M015). For the latter, set "Base orders per customer" under Simulation assumptions.
Simulation refused: `not_approved` / `stale_version` / `already_executed`	Approve the current version first; reload after an edit; each version runs once, so open its existing experiment.
`Activate.ps1` blocked	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Port already in use	Another uvicorn or Next.js is running. Close it or use `--port 8001` / `npm run dev -- -p 3001` (and update the URL/CORS settings).

16. Tests and quality checks

```
# backend (inside backend\, venv active)
python -m pytest                    # all backend tests (SQLite; PostgreSQL cases skip)
python -m ruff check .
python -m mypy app

# optional: also run the DB tests on a DISPOSABLE PostgreSQL database
$env:TEST_DATABASE_URL="postgresql+psycopg://nafapilot:nafapilot@localhost:5432/nafapilot_test"
python -m pytest tests/db tests/experiments

# frontend (inside frontend\)
npm run lint
npm run typecheck
npm test
npm run build
```

Suite	Last run (29 Oct 2026, this workspace)
Backend `pytest` (SQLite)	242 passed, 80 skipped (PostgreSQL variants without `TEST_DATABASE_URL`), 1 deselected
Backend DB + experiment tests on PostgreSQL 16 (M8)	175 passed, 2 skipped
Frontend `npm test`	25 passed (3 files)
ruff, mypy --strict, ESLint, TypeScript, `next build`	Clean

No test calls live Gemini. Frontend tests use real responses captured from the backend as fixtures.

17. Limitations and next step

- **Synthetic data and simulated execution only.** Results demonstrate the method; they are not evidence about real merchants.
- **Assumed order counts.** Predictions depend on the I and S assumptions (from the AI or the merchant). That's why there are three scenarios and why M8 measures against a holdout.
- **Screening statistics.** M4 alerts use nominal p-values; real orders vary more than the Poisson assumption.
- **Welch interval approximations**, missing-at-random assumption, redemptions treated as fixed in the profit interval; about 5% of null-effect runs show a "significant" result by chance.
- **One run per version**; no sequential testing.
- **API gaps noticed in M9:** no endpoint lists a merchant's recommendation runs (the campaign links back through its audit trail), and the campaign list doesn't include approval status (the dashboard loads details for the 20 most recent).
- **Interpretations to confirm:** post-discount margin definition and campaign-budget scope (chapter 8, M5).

Next: M10 — end-to-end tests, deployment (Render + Vercel + hosted PostgreSQL) and an offline fallback for the demo. Not started; it waits for review of M9.

18. Glossary

Incremental orders (I)	Orders caused by the campaign; they would not have happened without it.
Cannibalisation (S)	Orders that would have happened anyway but now get the discount. Their discount is pure cost.
Contribution profit	Revenue minus variable cost (and campaign cost), before fixed costs like rent.
Break-even	The number of incremental orders needed for $\Delta P \geq 0$.
Holdout	A randomly chosen group that does not get the offer, used as the comparison.
Confidence interval	A range of plausible values for the true effect. If it includes zero, the data can't rule out "no effect".
Input hash	A SHA-256 fingerprint of a campaign version's inputs. Approvals bind to it.
Idempotency key	A unique request ID. Sending the same request twice with the same key has the effect of sending it once.
Grounded explanation	AI text whose every number appears in the calculated records.
Migration	A versioned, reversible change to the database structure (Alembic).
Append-only	Rows can be added but never edited or deleted.
Benjamini–Hochberg	A method that limits the share of false alarms when many tests are run at once.
Welch's t-interval	A standard interval for the difference between two group means that doesn't assume equal variances.